ESPRIT - Se former autrement

Resume complet pour l'examen

Service et injection des dependances dans Angular 21

Prosit ConfTech : catalogue + reservations synchronisees | Signals + `inject()` | Architecture root singleton

Objectif : tout comprendre de zero, savoir quand utiliser quoi, et recracher le code propre le jour J.

Si l'examinateur demande...	Tu reponds...
C'est quoi un service ?	Classe metier deleguee : etat, API, logique. Composant = affichage seul.
Pourquoi DI ?	Pas de `new` manuel : l'injecteur cree et fournit. Testable, partage, synchro.
`providedIn: 'root'` ?	Singleton global, une seule instance partout. C'est ce qu'il faut pour les reservations.
`providers: [...]` dans composant ?	Instance locale detruite avec le composant. A EVITER pour un etat global.

Plan : 1. Service & SRP 2. Moteur DI 3. @Injectable / providedIn 4. Portee injecteurs 5. inject() 6. CLI 7. Services predefinis 8. Cas ConfTech (code) 8bis. Controle de saisie (HTML) 9. Fiches exam

1 La notion de service - la base

En Angular, un **Composant** doit se concentrer **exclusivement sur l'affichage (Vue) et l'interaction directe** (clic, input). Toute la logique **metier**, la **gestion d'etat global**, ou la **communication API** est deleguee a une classe dediee : le **Service**.

- **Separation des responsabilites (SRP) :** composants legers et lisibles. Un composant qui fetch + filtre + reserve = impossible a maintenir.
- **Reutilisabilite :** une meme logique/donnee partagee entre Header, Liste, Profil, Dashboard sans copier-coller.
- **Testabilite :** on remplace le vrai service par un **mock** (faux service) dans les tests unitaires. Avec `new MaClasse()` manuel, impossible.

Regle d'or a reciter : affichage dans les composants, donnees + logique reservation dans un ou plusieurs services. C'est exactement la consigne du prosit ConfTech.

2 Injection de dependances (DI) - le moteur

L'**Injection de Dependances** est un **Design Pattern** : une classe **ne cree pas elle-meme** les objets dont elle a besoin, elle **les recoit** d'une entite externe appelee **l'Injecteur**.

Schema mental a retenir

1. Tu declares le service avec `@Injectable({ providedIn: 'root' })` → 2. L'**Injecteur Angular** le **cree et le fournit** (singleton) → 3. Le **Composant** le **recoit** via `inject(MonService)`. Jamais de `new MonService()` dans un composant.

Piege classique ConfTech : `new ConferenceListComponent()` dans la Navbar = instance isolee, hors injecteur : pas de cycle de vie, pas de change detection, donnees desynchronisees, tests qui echouent. La correction = injecter le meme service singleton des deux cotes.

3 @Injectable() et providedIn - le coeur du cours

Pour qu'une classe TypeScript devienne injectable, on lui applique le decorateur `@Injectable()` importe de `@angular/core`. La propriete **providedIn** dit **ou** le service est enregistre, donc **combien d'instances** existent et **combien de temps** elles vivent.

```
@Injectable({
  providedIn: 'root'
})
export class MonService {}
```

3.1 'root' (recommande par defaut)

- **Ou :** RootInjector. **Comportement : Singleton global**, une seule instance pour toute l'app.
- **Tree-shakable : Oui.** Si jamais injecte, exclu du bundle final.
- **Cas d'usage :** etat global, auth, theme, client API HTTP, **ConferenceService + BookingService**.

3.2 'platform'

- **Ou :** PlatformInjector, un niveau au-dessus du root. **Partage entre plusieurs apps Angular** sur la meme page / meme fenetre (micro-frontends, bootstrapApplication multiples).
- **Cas d'usage :** logger partage, etat global inter micro-frontends. **Jamais pour ConfTech simple.**

3.3 'any'

- Une instance par **branche lazy-loaded**. Module principal = instance partagee ; chaque route lazy = sa propre instance.
- **Cas d'usage :** isoler un cache / etat specifique a un sous-domaine charge a la demande. **Dangereux pour des reservations globales** (compteurs differents par route).

3.4 Classe / Type<any> (perimetre cible)

Ex : `providedIn: AdminFeatureComponent`. Service lie au **cycle de vie et scope du type fourni**. Pour services fortement couples a un composant racine ou module specifique.

3.5 Non specifie (providedIn omis)

- **Comportement : pas d'enregistrement auto.** Obligation de l'ajouter dans `providers: [...]` : app.config.ts (bootstrapApplication), routes.ts, ou `@Component({ providers: [LocalStateService] })`.
- **Cas d'usage :** portee locale (recree a chaque instance du parent) ou substitution par env (`useClass` / `useFactory`). **Tree-shakable : Non.**

providedIn	Arbre DI	Portee	Tree-shakable ?
'root'	Root Injector	Singleton global	Oui
'platform'	Platform Injector	Inter-apps (micro-frontends)	Oui
'any'	Root / Lazy Route	1 instance par branche lazy	Oui
Non defini	providers: [] manuel	Depend du providers	Non

4 Portee des injecteurs (hierarchique)

L'injecteur Angular est **hierarchique**. Duree de vie et portee dependent de **l'endroit ou le service est fourni**.

Emplacement providers	Portee	Duree de vie
`{ providedIn: 'root' }`	**Globale (Singleton)**	Toute l'application
providers d'un **Composant**	**Locale** au composant + enfants	Detruit avec le composant
Dans une **Route**	**Scope de route**	Duree d'activation de la route

Reflexe exam : etat partage header + liste + profil + dashboard → **root**. Etat temporaire d'un formulaire local → providers du composant.

5 Comment injecter ? constructor vs inject()

Depuis les versions recentes, la methode **recommandee est** `inject()`, qui remplace l'injection par constructeur.

Ancienne approche (constructeur)

```
export class ProductListComponent {
  // Injection dans la signature du constructeur
  constructor(private productService: ProductService) {}
}
```

Approche moderne (Angular 21)

```
export class ProductListComponent {
  // Injection propre et fortement typee en une ligne
  private readonly productService = inject(ProductService);
}
```

Avantages de inject() - a reciter

- **Inference de type :** type deduit auto, pas de membre prive manuel.
- **Heritage facilite :** pas besoin de passer les dependances au `super()`.
- **Utilisable hors constructeur :** fonctions d'init, **Functional Guards**, resolvers.

6 Creer un service (CLI)

```
ng generate service nom-du-service
ng g s nom-du-service            # raccourci
ng g s services/nom-du-service   # dans sous-dossier
ng g s services/nom --skip-tests # sans .spec.ts
```

Genere le fichier du service + son test `.spec.ts`.

7 Services predefinis Angular

Service	Module	Usage
HttpClient	@angular/common/http	Appels API REST / HTTP
Router	@angular/router	Navigation entre pages
ActivatedRoute	@angular/router	Parametres d'URL
FormBuilder	@angular/forms	Construction rapide de formulaires

8 Cas ConfTech - de l'anti-pattern au propre (a coder)

8.1 Ce qui etait faux (AVANT)

- `conferences` en dur dans **ConferenceList** + logique de filtrage **dupliquee** dans Navbar.
- `new ConferenceListComponent()` dans la Navbar = 2 etats isoles, compteur jamais synchro.
- Violation SRP, maintenance impossible, tests KO.

8.2 Solution (APRES) - 2 services singletons

Service	Responsabilite unique	Signaux
ConferenceService (catalogue)	Donnees + filtre par sujet. Aucune reservation.	`signal[] + computed filtre`
BookingService (reservations)	Ids reserves + compteur + programme. Aucun filtrage duplique.	`signal[] + computed count`

Les deux en `@Injectable({ providedIn: 'root' })` : **une seule instance partagee** par Liste, Navbar, Profil, Dashboard.

Modele

```
export interface Conference {
  id: number; nom: string; sujet: string;
  date: string; lieu: string; places: number;
}
```

ConferenceService (catalogue)

```
@Injectable({ providedIn: 'root' })
export class ConferenceService {
  private readonly _conferences = signal<Conference[]>([...]);
  readonly filtreTexte = signal<string>('');
  readonly sujetChoisi = signal<string>('Tous');
  readonly placesMin = signal<number>(0);
  readonly conferences = this._conferences.asReadonly();
  readonly sujets = computed(() =>
    ['Tous', ...new Set(this._conferences().map(c => c.sujet))]);
  readonly filteredConferences = computed(() => {
    const t = this.filtreTexte().trim().toLowerCase();
    const s = this.sujetChoisi();
    const p = this.placesMin();
    return this._conferences().filter(c => {
      const matchTexte = !t || c.nom.toLowerCase().includes(t)
        || c.sujet.toLowerCase().includes(t) || c.lieu.toLowerCase().includes(t);
      return matchTexte && (s === 'Tous' || c.sujet === s) && c.places >= p;
    });
  });
  getById(id: number) {
    return this._conferences().find(c => c.id === id);
  }
  setFiltreTexte(v: string) { this.filtreTexte.set(v); }
  setSujet(v: string) { this.sujetChoisi.set(v); }
  setPlacesMin(v: number) { this.placesMin.set(v); }
  resetFiltres() {
    this.filtreTexte.set(''); this.sujetChoisi.set('Tous');
    this.placesMin.set(0);
  }
}
```

BookingService (reservations)

```
@Injectable({ providedIn: 'root' })
export class BookingService {
  private readonly conferenceService = inject(ConferenceService);
  private readonly _bookings = signal<Booking[]>([]);
  readonly bookedCount = computed(() => this._bookings().length);
  readonly totalPlaces = computed(() =>
    this._bookings().reduce((s, b) => s + b.places, 0));
  readonly bookedIds = computed(() =>
    this._bookings().map(b => b.conferenceId));
  readonly bookedConferences = computed(() =>
    this._bookings().map(b => this.conferenceService.getById(b.conferenceId))
      .filter(c => c !== undefined));
  isBooked(id: number) { return this.bookedIds().includes(id); }
  placesFor(id: number) {
    return this._bookings().find(b => b.conferenceId === id)?.places ?? 0;
  }
  book(id: number, places = 1) {
    if (this.isBooked(id)) return;
    this._bookings.update(l => [...l, { conferenceId: id, places }]);
  }
  cancel(id: number) {
    this._bookings.update(l => l.filter(b => b.conferenceId !== id));
  }
  toggle(id: number, places = 1) {
    this.isBooked(id) ? this.cancel(id) : this.book(id, places);
  }
}
```

ConferenceListComponent (fin, affichage seul)

```
export class ConferenceList {
  protected readonly conferenceService = inject(ConferenceService);
  protected readonly bookingService = inject(BookingService);
  onToggle(id: number) { this.bookingService.toggle(id); }
}
```

Template : `@for (c of conferenceService.filteredConferences(); track c.id)` + input filtre lie a `filtreSujet()`. Aucune donnee en dur.

Navbar / Header (meme instance !)

```
export class Header {
  protected readonly bookingService = inject(BookingService);
}
<!-- template -->
<span>Reservations : {{ bookingService.bookedCount() }}</span>
```

Un clic sur **Reserver** dans la liste → `_bookings.update()` → `bookedCount()` se recalcule → navbar + liste + profil se mettent a jour **en temps reel**, sans duplication.

8 bis Controle de saisie (HTML) - ngModel, Reactive Forms, validation

La **section HTML** du cours : comment on recupere ce que l'utilisateur saisit, et comment on affiche proprement les erreurs. Deux approches coexistent en Angular 21.

8 bis.1 Les 2 approches - tableau a connaitre

Approche	Module a importer	Lie a	Usage
Template-driven (ngModel)	`FormsModule`	Le template (directives)	Petits formulaires, **controle de saisie simple** (filtre, select)
Reactive-driven (FormBuilder)	`ReactiveFormsModule`	Le **code TS** (FormGroup)	Formulaires complexes, **validation**, **signals**

Regle : ngModel = liaison bidirectionnelle simple. Reactive Forms = quand tu veux **controler** le formulaire depuis le TypeScript (valider, desactiver, transformer les valeurs).

8 bis.2 Template-driven - ngModel (controle de saisie dans les filtres)

```
<label class="filter">
  Sujet
  <select [ngModel]="conferenceService.sujetChoisi()"
          (ngModelChange)="conferenceService.setSujet($event)"
          [ngModelOptions]="{ standalone: true }">
    @for (s of conferenceService.sujets(); track s) {
      <option [value]="s">{{ s }}</option>
    }
  </select>
</label>
```

- `[ngModel]` = **lecture** (source de verite = le signal du service).
- `(ngModelChange)` = **ecriture** (on remonte l'evenement vers le service).
- **Signal + ngModel = unidirectional** : le signal reste la source de verite, le template ne fait que refleter. C'est la version moderne du `[(ngModel)]`.
- `[ngModelOptions]="{ standalone: true }"` = **obligatoire** hors `<form>` : sinon Angular dit 'name is required'.
- `[ngValue]` (et non `[value]`) pour conserver le **type** (number) dans un select.

Erreur classique : mettre un input dans un `<form>` sans `name` et sans `standalone` → erreur NG01352.

8 bis.3 Reactive Forms - FormBuilder (formulaire utilisateur)

```
@Component({
  selector: 'app-userform',
  imports: [ReactiveFormsModule],
  templateUrl: './userform.html',
})
export class Userform {
  private readonly fb = inject(FormBuilder);

  readonly userForm = this.fb.nonNullable.group({
    name: ['', [Validators.required, Validators.minLength(2)]],
    password: ['', [Validators.required, Validators.minLength(6)]],
    email: ['', [Validators.required,
      Validators.pattern(/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/)]],
  });

  submitForm() { console.log(this.userForm.value); }
}
```

`nonNullable.group` → le **type** des controles est deduit (pas d'`| null`), les valeurs par defaut ne sont pas `null`.

8 bis.4 Template du formulaire

```
<form [formGroup]="userForm" (ngSubmit)="submitForm()"
      (ngReset)="validateForm()" novalidate>
  <label>
    Name
    <input type="text" formControlName="name" placeholder="Name">
  </label>
  <button type="submit">Submit</button>
  <button type="reset">Reset</button>
</form>
```

3 liens indispensables : `[formGroup]` (attache le groupe), `formControlName="name"` (attache chaque champ), `(ngSubmit)` (submit). `novalidate` desactive la validation native HTML : c'est **Angular** qui valide.

8 bis.5 Affichage des erreurs - LE PATRON A REPRENDRE

```
@for (champ of champs(); track champ.name) {
  <label class="champ">
    <span class="label">{{ champ.label }}</span>
    <input [type]="champ.type" [formControlName]="champ.name"
           [class.invalid]="champ.invalide && champ.touched" />

    @if (champ.invalide && champ.touched) {
      <span class="erreur">{{ champ.erreur }}</span>
    }
  </label>
}
```

Pourquoi `champ.invalide && champ.touched` ?

- **invalide** = le controle ne respecte pas les `Validators` (required, email, minLength...).
- **touched** = l'utilisateur a **quitte** le champ (blur). Sans touched, le formulaire s'afficherait **rouge et en erreur** des le premier affichage : mauvaise UX.
- Le **combo** = on ne reproche l'erreur que si l'utilisateur a vraiment essayé de saisir.

Les 4 etats d'un champ a connaitre (tableau d'examen) :

Etat	Signification	Bordure
pristine	Jamais modifie	Normale
touched	Focus quitte (blur)	Normale
invalid	Validators non respectes	Rouge **seulement si touched**
valid	OK	Verte / normale

8 bis.6 Construire l'etat de chaque champ (cote TS)

```
interface ChampEtat {
  name: ChampNom; label: string; type: string;
  invalide: boolean;   // control.invalid
  touched: boolean;    // control.touched
  erreur: string | null;
}

protected champs(): ChampEtat[] {
  return this.definitions.map(def => {
    const control = this.userForm.controls[def.name];
    return { ...def,
      invalide: control.invalid,
      touched: control.touched,
      erreur: this.erreurDe(control, def.messages) };
  });
}

private erreurDe(control: AbstractControl, messages: Record<string,string>) {
  if (!control.errors) return null;
  const cle = Object.keys(control.errors)[0];
  return messages[cle] ?? 'Valeur invalide.';
}
```

Logique : `Validators` en anglais dans le code → **messages en francais** dans un dictionnaire `messages`. Un seul endroit a traduire.

8 bis.7 Declencher la validation

```
submitForm() {
  this.submitted.set(true);
  if (this.userForm.invalid) {
    this.userForm.markAllAsTouched();  // affiche TOUTES les erreurs
    return;
  }
  console.log(this.userForm.value);
}

validateForm() {
  this.userForm.markAllAsTouched();
}
```

`markAllAsTouched()` = marque **tous** les champs comme touches => toutes les erreurs s'affichent d'un coup apres une soumission ratee. Indispensable.

8 bis.8 Validators a connaitre

Validator	Effet	Champs types
`Validators.required`	Valeur obligatoire	Tous
`Validators.email`	Format email	email
`Validators.minLength(n)`	Au moins n caracteres	password, name
`Validators.maxLength(n)`	Au plus n caracteres	name
`Validators.pattern(regex)`	Doit matcher la regex	email, tel
`Validators.min / max`	Borne numerique	age, places

8 bis.9 Reactive Forms + Signals (le combo moderne)

```
        // 1) le formulaire expose ses valeurs sous forme de signal
        readonly userFormValue = toSignal(
          this.userForm.valueChanges,
          { initialValue: this.userForm.getRawValue() }
        );

        // 2) un computed derive l'etat global a partir du formulaire
        readonly estValide = computed(() => this.userForm.valid);
        readonly estInscrit = signal(false);
```

- `toSignal(valueChanges)` transforme un Observable en **signal** : le formulaire alimentent directement un service singleton.
- On peut alors **ecrire dans un service** (inscription, panier) et relire l'etat partout avec `computed`.

Chaine complete : Formulaire (Reactive) → signal → `BookingService / UserService` (root) → navbar + dashboard → meme source de verite.

9 Fiches exam - pieges et reponses types

Question piege	Bonne reponse
Pourquoi pas `new Service()` ?	Hors injecteur : pas de singleton, pas de cycle de vie, pas de mock, desynchro. Toujours `inject()`.
`providedIn: 'root'` vs `providers: []` ?	root = global partage (reservations). providers composant = local detruit avec lui. Mauvais choix = 2 compteurs differents.
Quand `'any'` / `'platform'` ?	'any' = 1 instance par lazy-route (cache isole). 'platform' = partage inter micro-frontends. Ni l'un ni l'autre pour un panier global.
`signal` vs `computed` ?	signal = source modifiable (.set/.update). computed = derive auto, lecture seule, recalcule si dependance change.
Ou mettre le filtrage ?	Une seule fois dans ConferenceService (computed). Jamais copie dans 2 composants.
Comment tester ?	Mock du service : `TestBed.provide { provide: BookingService, useValue: fake }`. Composant leger = test simple.
ngModel ou Reactive Forms ?	ngModel = petit controle de saisie lie au template. Reactive Forms = validation et controle depuis le TS. Dans les 2 cas : `FormsModule` / `ReactiveFormsModule` a importer.
Pourquoi `champ.invalide && champ.touched` ?	invalide = validators non respectes. touched = l'utilisateur a quitte le champ. Sans touched, les erreurs s'affichent des le depart = mauvaise UX.
`standalone: true` ?	Obligatoire pour un ngModel hors `<form>`, sinon erreur NG01352 (name is required).
Comment afficher toutes les erreurs ?	`form.markAllAsTouched()` au submit : tous les champs passent touched -> toutes les erreurs apparaissent.

Checklist derniere minute

- Je sais ecrire `@Injectable({ providedIn: 'root' })` les yeux fermes.
- Je sais injecter avec `inject(X)` et expliquer les 3 avantages.
- Je sais reciter le tableau providedIn + portee + tree-shakable.
- Je sais dessiner le flux : signal → computed → 2 composants.
- Je sais ecrire `@if (champ.invalide && champ.touched)` et expliquer touched.
- Je sais choisir ngModel vs Reactive Forms et citer `markAllAsTouched()`.
- Je ne mets JAMAIS `new` ni de logique metier dans un composant.
- Commandes CLI : `ng g s services/booking --skip-tests`.

Question centrale ConfTech : **les services dedies + DI centralisent le catalogue et synchronisent les reservations** car le singleton root est l'unique source de verite reactive partagee par toute l'app.

Bon courage - relis les tableaux pages 3-4 et le code page 5, c'est 80% des points.